



AI & ML LAB
Deep Learning Lab

EXPERIMENTAL MANUAL

LIST OF EXPERIMENTS

1. Neural Network - Artificial Neural Network (ANN)
2. Neural Network - Convolutional Neural Network (CNN)
3. Neural Network - Recurrent Neural Network (RNN)
4. Natural Language Processing (NLP) - Part of Speech, Lemmatization and Stemming
5. Natural Language Processing (NLP) - Sentiment Analysis
6. Computer Vision - Face Detection & Body Detection
7. Computer Vision - Face Recognition
8. Computer Vision - Image Generation with Stable Diffusion
9. Transfer Learning - Feature Extraction
10. Transfer Learning - Fine Tuning

EXPERIMENT NO.: 1

NAME

Artificial Neural Network (ANN)

OBJECTIVE

To build an Artificial Neural Network with Stochastic Gradient Descent for the given dataset.

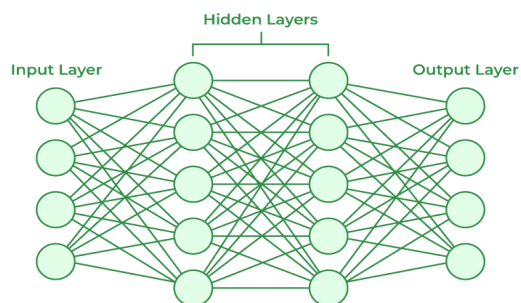
THEORY

Artificial Neural Network

Artificial Neural Networks are modeled after the **neurons** in the **human brain**. Artificial Neural Networks contain artificial neurons which are called **units**. These units are arranged in a series of layers that together constitute the whole Artificial Neural Network in a system. A layer can have only a **dozen units** or millions of units as this depends on how the complex neural networks will be required to learn the hidden patterns in the dataset.

Commonly, Artificial Neural Network has an input layer, an output layer as well as hidden layers. The **input layer** receives data from the outside world which the neural network needs to analyze or learn about. Then this data passes through one or multiple **hidden layers** that transform the input into data that is valuable for the **output layer**. Finally, the output layer provides an output in the form of a response of the Artificial Neural Networks to input data provided.

In the majority of neural networks, units are **interconnected** from one layer to another. Each of these connections has **weights** that determine the influence of one unit on another unit. As the data transfers from one unit to another, the neural network learns more and more about the data which eventually results in an output from the output layer.



The Neuron

The structures and operations of **human neurons** serve as the basis for artificial neural networks. It is also known as neural networks or neural nets. The input layer of an artificial neural network is the first layer, and it receives input from external sources and releases it to the hidden layer, which is the second

layer. In the hidden layer, each neuron receives input from the previous layer neurons, computes the weighted sum, and sends it to the neurons in the next layer. These connections are weighted means effects of the inputs from the previous layer are optimized more or less by assigning different-different weights to each input and it is adjusted during the training process by **optimizing these weights** for improved model performance.

Artificial neurons vs Biological neurons

The concept of artificial neural networks comes from biological neurons found in animal brains. So they share a lot of similarities in structure and function wise.

Structure

The structure of artificial neural networks is inspired by biological neurons. A biological neuron has a cell body or soma to process the impulses, dendrites to receive them, and an axon that transfers them to other neurons. The input nodes of artificial neural networks receive input signals, the hidden layer nodes compute these input signals, and the output layer nodes compute the final output by processing the hidden layer's results using activation functions.

Biological Neuron	Artificial Neuron
Dendrite	Inputs
Cell nucleus or Soma	Nodes
Synapses	Weights
Axon	Output

Synapses

Synapses are the links between biological neurons that enable the transmission of impulses from dendrites to the cell body. Synapses are the weights that join the one-layer nodes to the next-layer nodes in artificial neurons. The strength of the links is determined by the weight value.

Learning

In biological neurons, learning happens in the cell body nucleus or soma, which has a nucleus that helps to process the impulses. An action potential is produced and travels through the axons if the impulses are powerful enough to reach the threshold. This becomes possible by synaptic plasticity, which represents the ability of synapses to become stronger or weaker over time in reaction to changes in their activity. In artificial neural networks, backpropagation is a technique used for learning, which

adjusts the weights between nodes according to the error or differences between predicted and actual outcomes.

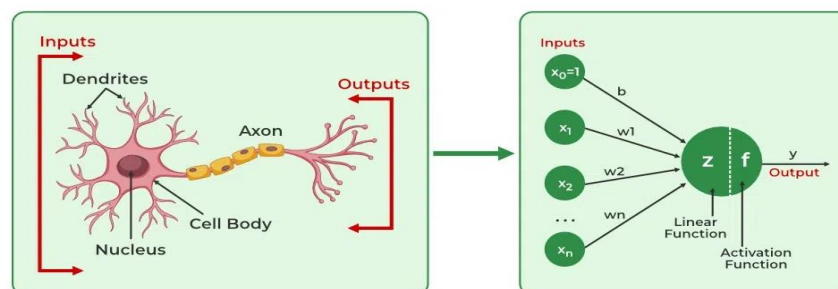
Biological Neuron	Artificial Neuron
Synaptic plasticity	Backpropagations

The Activation Function

In biological neurons, activation is the **firing rate** of the neuron which happens when the impulses are strong enough to reach the threshold. In artificial neural networks, A mathematical function known as an activation function maps the input to the output, and executes activations.

The purpose of an activation function is to introduce **non-linearity** into the model, allowing the network to learn and represent complex patterns in the data. Without non-linearity, a neural network would essentially behave like a linear regression model, regardless of the number of layers it has.

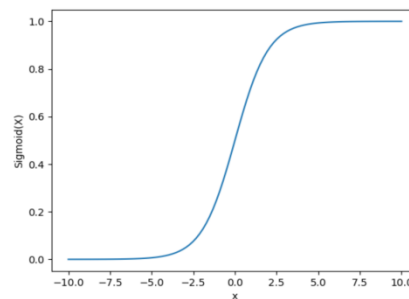
The activation function decides whether a neuron should be activated or not by calculating the **weighted sum** and further adding **bias** to it. The purpose of the activation function is to introduce non-linearity into the output of a neuron.



1. Sigmoid Function

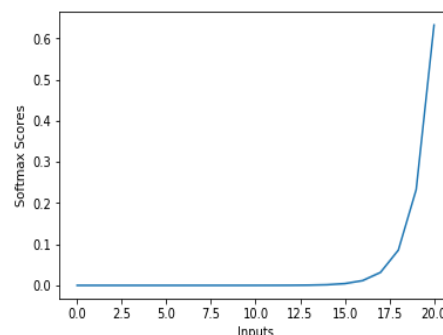
- **Sigmoid Function**-Activation functions in Neural Networks
- It is a function which is plotted as 'S' shaped graph.
- **Equation** : $A = 1/(1 + e^{-x})$
- **Nature** : Non-linear. Notice that X values lies between -2 to 2, Y values are very steep. This means, small changes in x would also bring about large changes in the value of Y.
- **Value Range** : 0 to 1

- **Uses** : Usually used in output layer of a binary classification, where result is either 0 or 1, as value for sigmoid function lies between 0 and 1 only so, result can be predicted easily to be 1 if value is greater than 0.5 and 0 otherwise.



2. RELU Function

- It Stands for **Rectified linear unit**. It is the most widely used activation function. Chiefly implemented in hidden layers of Neural network.
- **Equation** :- $A(x) = \max(0, x)$. It gives an output x if x is positive and 0 otherwise.
- **Value Range** :- $[0, \infty)$
- **Nature** :- non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
- **Uses** :- ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.



Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a variant of the Gradient Descent algorithm that is used for **optimizing** machine learning models. It addresses the computational inefficiency of traditional Gradient Descent methods when dealing with large datasets in machine learning projects.

In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters. This random selection introduces randomness into the optimization process, hence the term “**stochastic**” in Stochastic Gradient Descent.

The advantage of using SGD is its **computational efficiency**, especially when dealing with large datasets. By using a single example or a small batch, the computational cost per iteration is significantly reduced compared to traditional Gradient Descent methods that require processing the entire dataset.

Note:- Refer the slide given in the drive link.

Backpropagation

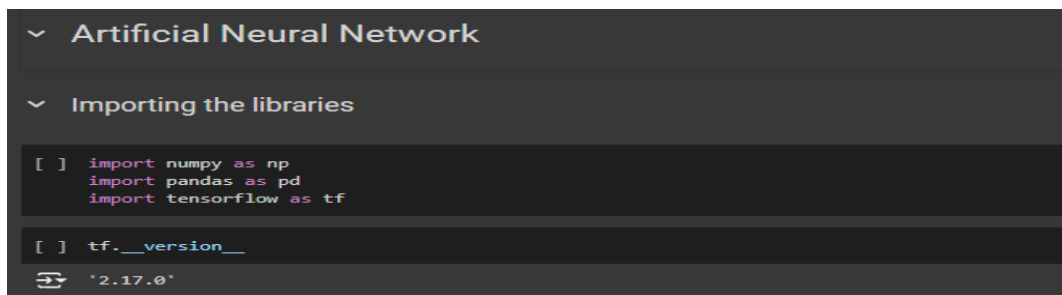
Backpropagation is a widely used algorithm for training **feedforward** neural networks. It computes the gradient of the loss function with respect to the **network weights**. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to **minimize loss**; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and **iterating backward** from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It compares generated output to the desired output and generates an **error report** if the result does not match the generated output vector. Then it adjusts the weights according to the **bug report** to get your desired output.

PROCEDURE

1. Click on New notebook in Drive and Create a new project file.
2. Download the **dataset : Churn_Modelling.csv** on your device using the given drive link.
3. Edit the title of the File by clicking on the '**Untitled0.ipynb**' text.
4. Create a new Text cell by clicking on the '**+ Text**' symbol.
5. Create a new code cell by clicking on the '**+ Code**' symbol.
6. Import the necessary libraries like :-
 - a) **Numpy** - which is used for working with arrays.
 - b) **Pandas** - which is used for data manipulation and analysis.
 - c) **Tensorflow** - which is used to develop models for various tasks, including neural networks, natural language processing, image recognition, etc.
7. Make sure you are using the latest version of Tensorflow. To check the version, type '**tf.__version__**' as below.



```
Artificial Neural Network

Importing the libraries

[ ] import numpy as np
import pandas as pd
import tensorflow as tf

[ ] tf.__version__

'2.17.0'
```

8. Import the dataset that you have downloaded by clicking on the File icon and selecting the

file.

```

Part 1 - Data Preprocessing
Importing the dataset

[ ] dataset = pd.read_csv('Churn_Modelling.csv')
    x = dataset.iloc[:, 3:-1].values
    y = dataset.iloc[:, -1].values

[ ] print(X)
[[619 'France' 'Female' ... 1 1 101348.88]
 [608 'Spain' 'Female' ... 0 1 112542.58]
 [502 'France' 'Female' ... 1 0 113931.57]
 ...
 [709 'France' 'Female' ... 0 1 42085.58]
 [772 'Germany' 'Male' ... 1 0 92888.52]
 [792 'France' 'Female' ... 1 0 38190.78]]

[ ] print(y)
[1 0 1 ... 1 1 0]

```

9. Perform Label Encoding on the **'Gender'** Column using Scikit-learn library to convert it into a set of binary values.

```

Encoding categorical data
Label Encoding the "Gender" column

[ ] from sklearn.preprocessing import LabelEncoder
    le = LabelEncoder()
    X[:, 2] = le.fit_transform(X[:, 2])

[ ] print(X)
[[619 'France' 0 ... 1 1 101348.88]
 [608 'Spain' 0 ... 0 1 112542.58]
 [502 'France' 0 ... 1 0 113931.57]
 ...
 [709 'France' 0 ... 0 1 42085.58]
 [772 'Germany' 1 ... 1 0 92888.52]
 [792 'France' 0 ... 1 0 38190.78]]

```

10. To Encode the Categorical data perform One Hot Encoding on the **'Geography'** Column using the Scikit-learn library on the State Column and convert it into a set of binary values.

```

One Hot Encoding the "Geography" column

[ ] from sklearn.compose import ColumnTransformer
    from sklearn.preprocessing import OneHotEncoder
    ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [1])], remainder='passthrough')
    X = np.array(ct.fit_transform(X))

[ ] print(X)
[[1.0 0.0 0.0 ... 1 1 101348.88]
 [0.0 0.0 1.0 ... 0 1 112542.58]
 [1.0 0.0 0.0 ... 1 0 113931.57]
 ...
 [1.0 0.0 0.0 ... 0 1 42085.58]
 [0.0 1.0 0.0 ... 1 0 92888.52]
 [1.0 0.0 0.0 ... 1 0 38190.78]]

```

11. Split the dataset into a training and testing set to avoid **overfitting** the model.

✓ Splitting the dataset into the Training set and Test set

```
[ ] from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

✓ Feature Scaling

```
[ ] from sklearn.preprocessing import StandardScaler
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)
```

12. Perform Feature Scaling to **standardize** the independent features present in the data in a fixed range using **Scikit-learn** library.

✓ Feature Scaling

```
[ ] from sklearn.preprocessing import StandardScaler
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)
```

13. To start building the Artificial Neural Network (ANN), initialize the model using **Keras** library which is integrated with **Tensorflow** using the Sequential module.
14. Add the first input and hidden layer using the **'layers'** module of the Keras library. Input the activation function as **'RELU'** in the parameter.
15. Add the second hidden layer using the **'layers'** module of the Keras library. Input the activation function as **'RELU'** in the parameter.
16. Add the output layer using the **'layers'** module of the Keras library. Input the activation function as **'Sigmoid'** in the parameter.

✓ Part 2 - Building the ANN

✓ Initializing the ANN

```
[ ] ann = tf.keras.models.Sequential()
```

✓ Adding the input layer and the first hidden layer

```
[ ] ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
```

✓ Adding the second hidden layer

```
[ ] ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
```

✓ Adding the output layer

```
[ ] ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

17. Compile the Artificial Neural Network (ANN) using the '**Adam**' optimizer and '**binary_crossentropy**' loss.
18. Train the Artificial Neural Network (ANN) for a batch size of '**32**' and '**100**' epochs.

```

  ▾ Part 3 - Training the ANN

  ▾ Compiling the ANN

  [ ] ann.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])

  ▾ Training the ANN on the Training set

  ▶ ann.fit(X_train, y_train, batch_size = 32, epochs = 100)
  Epoch 59/100 0s 210us/step - accuracy: 0.8658 - loss: 0.3245
  250/250
  Epoch 60/100 0s 884us/step - accuracy: 0.8666 - loss: 0.3245
  250/250
  Epoch 61/100 0s 933us/step - accuracy: 0.8656 - loss: 0.3255
  250/250
  Epoch 62/100 0s 941us/step - accuracy: 0.8629 - loss: 0.3368
  250/250
  Epoch 63/100 0s 915us/step - accuracy: 0.8627 - loss: 0.3363
  250/250
  Epoch 64/100 0s 963us/step - accuracy: 0.8693 - loss: 0.3219
  250/250
  Epoch 65/100 0s 953us/step - accuracy: 0.8730 - loss: 0.3135
  250/250
  Epoch 66/100 0s 848us/step - accuracy: 0.8654 - loss: 0.3310
  250/250
  Epoch 66/100

```

OUTPUT

1. Predict the result of a single observation.

```

  ▾ Part 4 - Making the predictions and evaluating the model

  ▾ Predicting the result of a single observation

  Homework
  Use our ANN model to predict if the customer with the following informations will leave the bank:
  Geography: France
  Credit Score: 600
  Gender: Male
  Age: 40 years old
  Tenure: 3 years
  Balance: $ 60000
  Number of Products: 2
  Does this customer have a credit card? Yes
  Is this customer an Active Member: Yes
  Estimated Salary: $ 50000
  So, should we say goodbye to that customer?

```

```

  Solution

  [ ] print(ann.predict(sc.transform([[1, 0, 0, 600, 1, 40, 3, 60000, 2, 1, 1, 50000]])) > 0.5)

  1/1 0s 42ms/step
  [[False]]

  Therefore, our ANN model predicts that this customer stays in the bank!

  Important note 1: Notice that the values of the features were all input in a double pair of square brackets. That's because the "predict" method always expects a 2D array as the format of its inputs. And putting our values into a double pair of square brackets makes the input exactly a 2D array.

  Important note 2: Notice also that the "France" country was not input as a string in the last column but as "1, 0, 0" in the first three columns. That's because of course the predict method expects the one-hot-encoded values of the state, and as we see in the first row of the matrix of features X, "France" was encoded as "1, 0, 0". And be careful to include these values in the first three columns, because the dummy variables are always created in the first columns.

```

2. Predict the Testing set results.

```
▼ Predicting the Test set results

y_pred = ann.predict(X_test)
y_pred = (y_pred > 0.5)
print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))

63/63 ————— 0s 781us/step
[[0 0]
 [0 1]
 [0 0]
 ...
 [0 0]
 [0 0]
 [0 0]]
```

3. Make a Confusion Matrix to check the **accuracy** of the model.

```
▼ Making the Confusion Matrix

from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test, y_pred)

[[1514  81]
 [ 204 201]]
0.8575
```

4. Run all the code cells to make sure there are no errors.

EXPERIMENT NO.: 2

NAME

Convolutional Neural Network (CNN)

OBJECTIVE

To build a Convolutional Neural Network for the given dataset.

THEORY

A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.

When it comes to Machine Learning, Artificial Neural Networks perform really well. Neural Networks are used in various datasets like images, audio, and text. Different types of Neural Networks are used for different purposes, for example for predicting the sequence of words we use Recurrent Neural Networks more precisely an LSTM, similarly for **image classification** we use Convolution Neural networks.

In a regular Neural Network there are **three** types of layers:

1. **Input Layers:** It's the layer in which we give input to our model. The number of neurons in this layer is equal to the **total number of features** in our data (number of pixels in the case of an image).
2. **Hidden Layer:** The input from the Input layer is then fed into the hidden layer. There can be many hidden layers depending on our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by **matrix multiplication** of the output of the previous layer with learnable weights of that layer and then by the addition of learnable biases followed by activation function which makes the network nonlinear.
3. **Output Layer:** The output from the hidden layer is then fed into a logistic function like **sigmoid** or **softmax** which converts the output of each class into the probability score of each class.

Convolutional Neural Network

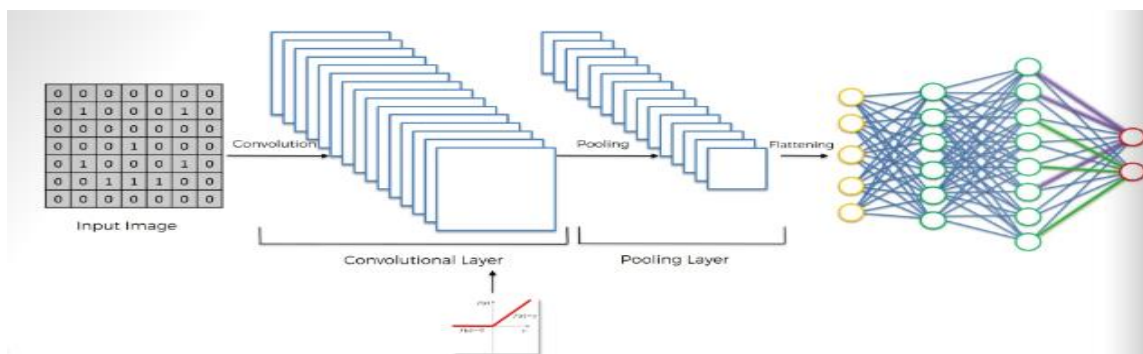
Convolutional Neural Network (CNN) is the extended version of Artificial Neural Networks (ANN) which is predominantly used to extract the feature from the **grid-like matrix** dataset. For example visual datasets like images or videos where data patterns play an extensive role.

CNN architecture

Convolutional Neural Network consists of multiple layers like the Input layer, Convolutional layer, Pooling layer, and Fully connected layers.

The Convolutional layer applies filters to the input image to extract features, the Pooling layer downsamples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

1. **Input Layer:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
2. **Convolutional Layer:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2 , 3×3 , or 5×5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension $32 \times 32 \times 12$.
3. **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are RELU: $\max(0, x)$, Tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimensions $32 \times 32 \times 12$.
4. **Pooling Layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are max pooling and average pooling. If we use a max pool with 2×2 filters and stride 2, the resultant volume will be of dimension $16 \times 16 \times 12$.



5. **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
6. **Fully Connected Layer:** It takes the input from the previous layer and computes the final classification or regression task.
7. **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

Data Augmentation

Data augmentation is the process of increasing the **amount** and **diversity** of data. We do not collect new data, rather we transform the already present data. Data augmentation is an integral process in deep learning, as in deep learning we need large amounts of data and in some cases it is not feasible to collect thousands or millions of images, so data augmentation comes to the rescue. It helps us to **increase** the **size** of the dataset and introduce **variability** in the dataset.

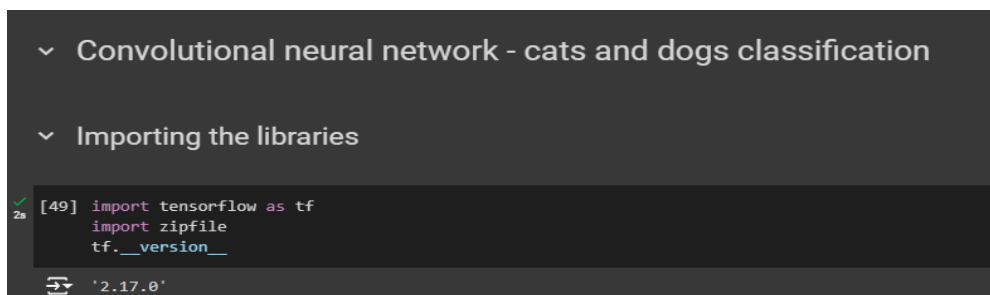
The most commonly used operations are :-

1. **Rotation:** Rotation operation as the name suggests, just rotates the image by a certain specified degree.
2. **Shearing:** Shearing is also used to transform the orientation of the image.
3. **Zooming:** Zooming operation allows us to either zoom in or zoom out.
4. **Cropping:** Cropping allows us to crop the image or select a particular area from an image.
5. **Flipping:** Flipping allows us to flip the orientation of the image. We can use horizontal or vertical flip.
6. **Changing the brightness level:** This feature helps us to combat illumination changes. You can encounter a scenario where most of your dataset comprises of images having a similar brightness level.

Example :- Collecting the images of employees entering the office, by augmenting the images we make sure that our model is **robust** and is able to detect the person even in **different surroundings**.

PROCEDURE

1. Click on New notebook in Drive and Create a new project file.
2. Download the **dataset : dataset_cat_dog.zip** on your device using the given drive link.
3. Edit the title of the File by clicking on the '**Untitled0.ipynb**' text.
4. Create a new Text cell by clicking on the '**+ Text**' symbol.
5. Create a new code cell by clicking on the '**+ Code**' symbol.
6. Import the necessary libraries like :-
 - a) **Tensorflow** - which is used to develop models for various tasks, including neural networks, natural language processing, image recognition, etc.
 - b) **Zipfile** - which is used for manipulating, reading and writing Zip files.
7. Make sure you are using the latest version of Tensorflow. To check the version, type '**tf.__version__**' as below.



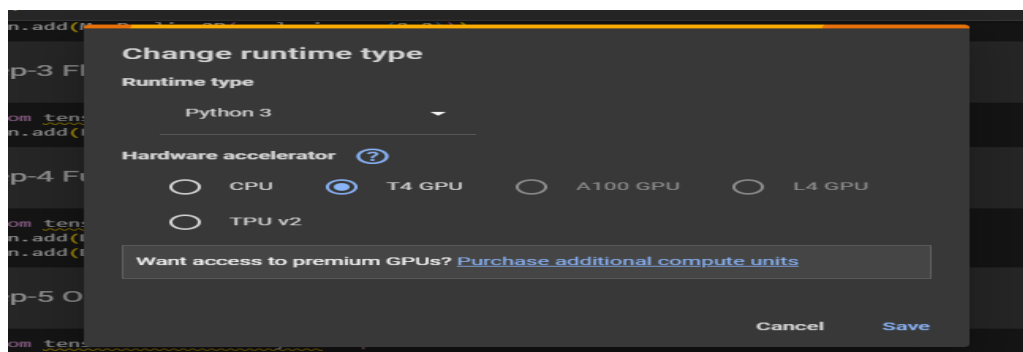
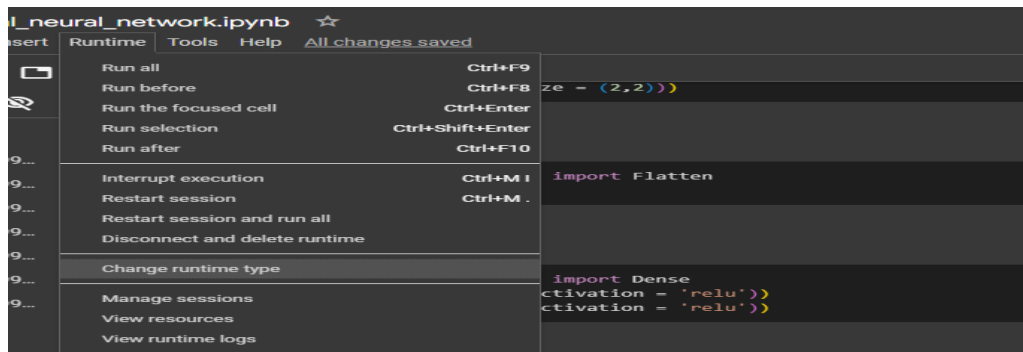
```
Convolutional neural network - cats and dogs classification

Importing the libraries

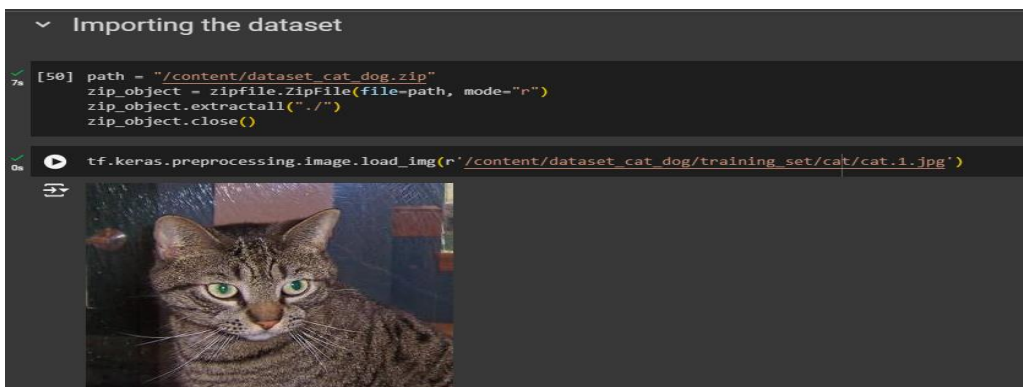
[49] import tensorflow as tf
import zipfile
tf.__version__

'2.17.0'
```

8. Convolutional Neural Network (CNN) takes a large amount of time to Train on the CPU. So to accelerate the Training, click on the symbol '**Change the Runtime**' and select '**T4GPU**' as shown below.



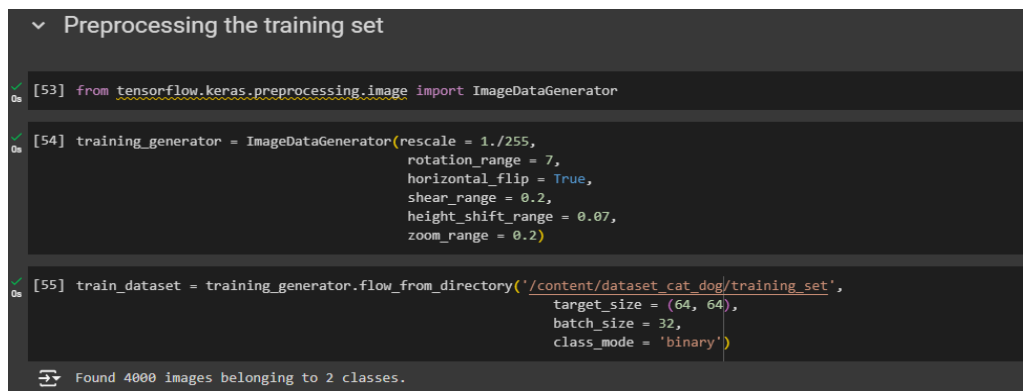
9. Import the dataset by providing the path to the zip object. Extract all the files from the dataset.
10. Load the first '**cat**' image from the Training set by using the '**preprocessing**' module of the Keras library and providing the path to it.



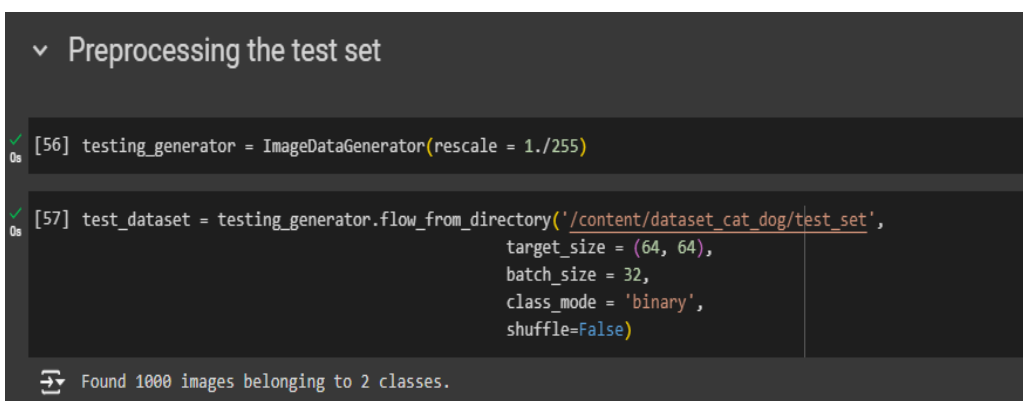
11. Likewise, load the first '**dog**' image from the Training set by using the '**preprocessing**' module of the Keras library and providing the path to it.



12. To avoid overfitting, Perform **Data Augmentation** by applying some transformations like geometrical, zooms, rotations and flips on the images of the training set using **ImageDataGenerator** class.



13. Specify the target size, batch size and the class mode of both the training and testing dataset.



14. To start building the Convolutional Neural Network (CNN), initialize the model using the **'Sequential'** module of the Keras library.
15. To perform Convolution on the input layer, import **'Conv2D'** from the Layers module of the Keras library. Input the activation function as **'RELU'** in the parameter.


```

  ▾ Part-2 Building the CNN

  ▾ Initialising the CNN

  [58] from tensorflow.keras.models import Sequential
       cnn = tf.keras.models.Sequential()

  ▾ Step-1 Convolution

  [59] from tensorflow.keras.layers import Conv2D
       cnn.add(Conv2D(32, (3,3), input_shape = (64, 64, 3), activation = 'relu'))

  /usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pa
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

16. Perform Pooling using **'MaxPooling2D'** from the Layers module. Input the pool size in the parameter.
17. Add a second convolutional layer using **'Conv2D'** and apply pooling to it using **'MaxPooling2D'**.

```

  ▾ Step-2 Pooling

  [60] from tensorflow.keras.layers import MaxPooling2D
       cnn.add(MaxPooling2D(pool_size = (2,2)))

  ▾ Adding a second convolutional layer

  [61] from tensorflow.keras.layers import Conv2D, MaxPooling2D
       cnn.add(Conv2D(32, (3,3), input_shape = (64, 64, 3), activation = 'relu'))
       cnn.add(MaxPooling2D(pool_size = (2,2)))

```

18. Perform Flattening of layers using **'Flatten'** from the Layers module.
19. Perform the Full Connection by connecting it to a Neural Network using **'Dense'** from the Layers module. Input the activation function as **'RELU'** in the parameter.
20. Connect an Output layer by using **'Dense'** from the Layers module. Input the activation function as **'Sigmoid'** in the parameter.

```

  ▾ Step-3 Flattening

  [62] from tensorflow.keras.layers import Flatten
       cnn.add(Flatten())

  ▾ Step-4 Full Connection

  [63] from tensorflow.keras.layers import Dense
       cnn.add(Dense(units = 128, activation = 'relu'))
       cnn.add(Dense(units = 128, activation = 'relu'))

  ▾ Step-5 Output Layer

  [64] from tensorflow.keras.layers import Dense
       cnn.add(Dense(units = 1, activation = 'sigmoid'))

```

21. Compile the Convolutional Neural Network (CNN) using the **'Adam'** optimizer and **'binary_crossentropy'** loss.

22. Train the Convolutional Neural Network (CNN) for **'100'** epochs. This cell might take longer to execute.

```
[65] cnn.compile(optimizer = 'adam', loss = 'binary_crossentropy',
                 metrics = ['accuracy'])
```

▼ Training the CNN on the training set

```
[66] cnn.fit(train_dataset, epochs = 100)
```

```
Epoch 49/100 42s 231ms/step - accuracy: 0.9264 - loss: 0.1707
125/125
Epoch 50/100 29s 230ms/step - accuracy: 0.9326 - loss: 0.1698
125/125
Epoch 51/100 30s 232ms/step - accuracy: 0.9301 - loss: 0.1853
125/125
Epoch 52/100 40s 225ms/step - accuracy: 0.9271 - loss: 0.1889
125/125
```

OUTPUT

1. Make predictions on the Testing data.
2. Predict the Testing set results.

```
✓ Evaluating the CNN

[67] predictions = cnn.predict(test_dataset)
3s 32/32 75ms/step

predictions
0s
[9.51149559e-05],
[9.11753705e-06],
[1.44880441e-02],
[1.33458688e-03],
[8.46282482e-01],
[3.65986526e-02],
[3.50826718e-02],
[3.02177817e-02],
[1.86294457e-03],
[9.9961217e-01],
[2.44260463e-03],
[9.99973476e-01],
[6.04968662e-08],
[9.99854684e-01],
```

- Convert the Testing set values to Boolean values by setting a threshold of **0.5**, where the values greater than 0.5 are **True** and the values less than 0.5 are **False**.

```
[69] predictions = (predictions > 0.5)
      predictions
```

```
[False],
[False],
[False],
[False],
[ True],
[False],
[False],
[False],
[False],
[ True],
[False],
[ True],
[False],
[ True],
[False],
[ True],
```

4. Make a Confusion Matrix to check the **accuracy** of the model.

EXPERIMENT NO.: 3

NAME

Recurrent Neural Network (RNN)

OBJECTIVE

To build a Recurrent Neural Network (RNN) using Long Short-Term Memory (LSTM) for the given dataset.

THEORY

Today, different Machine Learning techniques are used to handle different types of data. One of the most difficult types of data to handle and the forecast is sequential data. **Sequential data** is different from other types of data in the sense that while all the features of a typical dataset can be assumed to be order-independent, this cannot be assumed for a sequential dataset.

To handle such type of data, the concept of Recurrent Neural Networks was conceived. It is different from other Artificial Neural Networks in its structure. While other networks “travel” in a linear direction during the feed-forward process or the back-propagation process, the Recurrent Network follows a recurrence relation instead of a feed-forward pass and uses **Back-Propagation** through time to learn.

Recurrent Neural Network

Recurrent Neural Network (RNN) is a type of Neural Network where the output from the previous step is **fed** as input to the current step.

In traditional neural networks, all the inputs and outputs are **independent** of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a **Hidden Layer**.

The main and most important feature of RNN is its Hidden state, which remembers some information about a **sequence**. The state is also referred to as **Memory State** or **Memory Cell** since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.

Recurrent Neuron

The fundamental processing unit in a Recurrent Neural Network (RNN) is a Recurrent Unit, which is not explicitly called a “**Recurrent Neuron**”. This unit has the unique ability to maintain a hidden state, allowing the network to capture sequential dependencies by remembering previous inputs while processing. Long Short-Term Memory (LSTM) version improves the RNN’s ability to handle **long-term dependencies**.

Long-Term Dependencies

One of the appeals of RNNs is the idea that they might be able to connect **previous information** to the present task, such as using previous video frames might inform the understanding of the present frame.

Sometimes, we only need to look at **recent information** to perform the present task. For example, consider a language model trying to predict the next word based on the previous ones. In such cases, where the gap between the relevant information and the place that it's needed is small, RNNs can learn to use the **past information**.

But there are also cases where we need more **context**. It's entirely possible for the gap between the relevant information and the point where it is needed to become very large. Unfortunately, as that gap grows, RNNs become **unable** to learn to connect the information. This is where **Long Short-Term Memory (LSTM)** comes into picture.

Vanishing Gradient

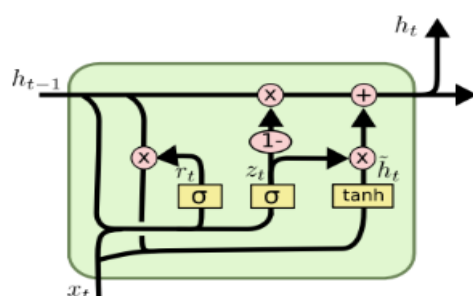
During backpropagation through time, gradients can become extremely small as they are multiplied through the chain of recurrent connections, causing the model to have difficulty learning dependencies that are separated by many time steps.

Long Short-Term Memory (LSTM)

Long Short Term Memory networks – usually just called “LSTMs” – are a special kind of RNN, capable of learning **long-term dependencies**. They were introduced by Hochreiter & Schmidhuber (1997), and were refined and popularized by many people in following work. They work tremendously well on a large variety of problems, and are now widely used.

In concept, an LSTM recurrent unit tries to “**remember**” all the past knowledge that the network is seen so far and to “**forget**” irrelevant data. This is done by introducing different activation function layers called “**gates**” for different purposes. Each LSTM recurrent unit also maintains a vector called the **Internal Cell State** which conceptually describes the information that was chosen to be retained by the previous LSTM recurrent unit.

LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their **default behavior**, not something they struggle to learn.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

1. **Memory Cell** - Remembering and Forgetting information from previous output which is controlled by the following three gates - Forget Gate, Input Gate, Output Gate.

2. **Forget Gate** - Deciding what information gets transferred and what information is forgotten using Sigmoid activation function (0 to 1) to eliminate change in context. The information that is no longer useful in the cell state is removed with the forget gate.
3. **Input Gate** - Adding information to the memory cell using the tanh activation function (1 to -1). The addition of useful information to the cell state is done by the input gate. First, the information is regulated using the sigmoid function and filter the values to be remembered similar to the Forget gate.
4. **Output Gate** - Passing output to next cell. The task of extracting useful information from the current cell state to be presented as output is done by the output gate. First, a vector is generated by applying tanh function on the cell. Then, the information is regulated using the sigmoid function and filtered by the values to be remembered.

Applications of LSTM

Some of the famous applications of LSTM includes:

1. **Language Modeling**: LSTMs have been used for natural language processing tasks such as language modeling, machine translation, and text summarization. They can be trained to generate coherent and grammatically correct sentences by learning the dependencies between words in a sentence.
2. **Speech Recognition**: LSTMs have been used for speech recognition tasks such as transcribing speech to text and recognizing spoken commands. They can be trained to recognize patterns in speech and match them to the corresponding text.
3. **Time Series Forecasting**: LSTMs have been used for time series forecasting tasks such as predicting stock prices, weather, and energy consumption. They can learn patterns in time series data and use them to make predictions about future events.
4. **Anomaly Detection**: LSTMs have been used for anomaly detection tasks such as detecting fraud and network intrusion. They can be trained to identify patterns in data that deviate from the norm and flag them as potential anomalies.
5. **Recommender Systems**: LSTMs have been used for recommendation tasks such as recommending movies, music, and books. They can learn patterns in user behavior and use them to make personalized recommendations.
6. **Video Analysis**: LSTMs have been used for video analysis tasks such as object detection, activity recognition, and action classification. They can be used in combination with other neural network architectures, such as Convolutional Neural Networks (CNNs), to analyze video data and extract useful information.

PROCEDURE

1. Click on New notebook in Drive and Create a new project file.
2. Download the datasets : **Google_train_data.csv** and **Google_test_data.csv** on your device using the given drive link.
3. Edit the title of the File by clicking on the '**Untitled0.ipynb**' text.

4. Create a new Text cell by clicking on the '+ Text' symbol.
5. Create a new code cell by clicking on the '+ Code' symbol.
6. Import the necessary libraries like :-
 - a) **Numpy** - which is used for working with arrays.
 - b) **Pandas** - which is used for data manipulation and analysis.
 - d) **Tensorflow** - which is used to develop models for various tasks, including neural networks, natural language processing, image recognition, etc.
7. Make sure you are using the latest version of Tensorflow. To check the version, type '`tf.__version__`' as below.

Importing the libraries

```
[1] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
tf.__version__
```

'2.17.0'

8. Import the **Train** dataset that you have downloaded by clicking on the File icon and selecting the file.

Importing the training dataset

```
data = pd.read_csv('Google_train_data.csv')
data.head()
```

	Date	Open	High	Low	Close	Volume
0	1/3/2012	325.25	332.83	324.97	663.59	7,380,500
1	1/4/2012	331.27	333.87	329.08	666.45	5,749,400
2	1/5/2012	329.83	330.75	326.89	657.21	6,590,300
3	1/6/2012	328.34	328.77	323.68	648.24	5,405,900
4	1/9/2012	322.04	322.29	309.46	620.76	11,688,800

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1258 entries, 0 to 1257
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        1258 non-null   object
1   Open        1258 non-null   float64
2   High        1258 non-null   float64
3   Low         1258 non-null   float64
4   Close       1258 non-null   object
5   Volume      1258 non-null   object
dtypes: float64(3), object(3)
memory usage: 59.1+ KB
```

9. Perform preprocessing on the data by Cleaning the '**Close**' column and removing all the missing values.

```

Cleaning the training dataset

[5] data["Close"] = pd.to_numeric(data.Close, errors='coerce')
data = data.dropna()
trainData = data.iloc[:,4:5].values

data.info()

<class 'pandas.core.frame.DataFrame'>
Index: 1149 entries, 0 to 1257
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype  
---  -
0    Date        1149 non-null    object  
1    Open        1149 non-null    float64  
2    High        1149 non-null    float64  
3    Low         1149 non-null    float64  
4    Close       1149 non-null    float64  
5    Volume      1149 non-null    object  
dtypes: float64(4), object(2)
memory usage: 62.8+ KB

```

10. Perform Feature Scaling to **standardize** the independent features present in the data in a fixed range using the '**MinMaxScaler**' in a feature range of (0 to 1) of the **Scikit-learn** library.

```

Feature Scaling

from sklearn.preprocessing import MinMaxScaler
sc = MinMaxScaler(feature_range=(0,1))
trainData = sc.fit_transform(trainData)
trainData.shape

(1149, 1)

```

11. Reshape the input to be in **Timesteps** which is required for Long Short-Term Memory (LSTM). Then, convert the data from a pandas dataframe to a numpy array.

```

Adding the Timesteps

[8] X_train = []
    y_train = []

    for i in range(60,1149): #60 : timestep // 1149 : length of the data
        X_train.append(trainData[i-60:i,0])
        y_train.append(trainData[i,0])

    X_train,y_train = np.array(X_train),np.array(y_train)

[9] X_train = np.reshape(X_train,(X_train.shape[0],X_train.shape[1],1)) #adding the batch_size axis
    X_train.shape

(1089, 60, 1)

```

12. To start building the Recurrent Neural Network (RNN), initialize the model using the '**Sequential**' module of the Keras library.

```

Building the RNN

Initializing the RNN

[10] from keras.models import Sequential
     rnn = Sequential()

```


13. Add the LSTM and Dropout using the **'Layers'** module of the Keras library.

```

  ▾ Adding the LSTM

  0s  ▶ from keras.layers import LSTM, Dropout
      rnn.add(LSTM(units=100, return_sequences = True, input_shape =(X_train.shape[1],1)))
      rnn.add(Dropout(0.2))

      rnn.add(LSTM(units=100, return_sequences = True))
      rnn.add(Dropout(0.2))

      rnn.add(LSTM(units=100, return_sequences = True))
      rnn.add(Dropout(0.2))

      rnn.add(LSTM(units=100, return_sequences = False))
      rnn.add(Dropout(0.2))

  /usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an `input_sh
    super().__init__(**kwargs)
  
```

14. Add the Output layer using the **'Dense'** module of the Keras library.
15. Compile the Recurrent Neural Network (RNN) model using the **'Adam'** optimizer and **'Mean squared error'** loss.

```

  ▾ Adding the Output layer

  0s  [12] from keras.layers import Dense
      rnn.add(Dense(units =1))

  ▾ Compiling the RNN model

  0s  [13] rnn.compile(optimizer='adam',loss="mean_squared_error")
  
```

16. Train the Recurrent Neural Network (RNN) model on the Training set for a batch size of **'32'** and **'20'** epochs.

```

  ▾ Training the RNN model on the Training set

  3m  [14] hist = rnn.fit(X_train, y_train, epochs = 20, batch_size = 32, verbose=2)

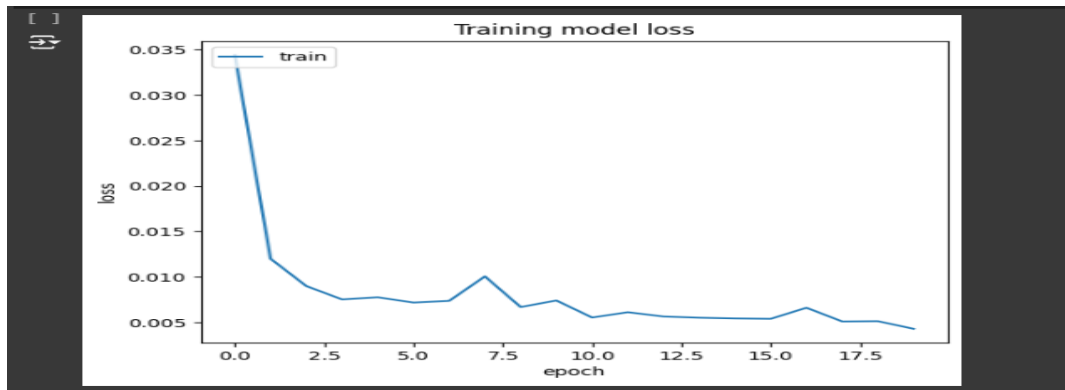
  Epoch 1/20
  35/35 - 17s - 479ms/step - loss: 0.0370
  Epoch 2/20
  35/35 - 12s - 330ms/step - loss: 0.0107
  Epoch 3/20
  35/35 - 11s - 302ms/step - loss: 0.0112
  Epoch 4/20
  35/35 - 7s - 194ms/step - loss: 0.0099
  Epoch 5/20
  35/35 - 13s - 357ms/step - loss: 0.0081
  Epoch 6/20
  35/35 - 10s - 275ms/step - loss: 0.0080
  Epoch 7/20
  35/35 - 7s - 194ms/step - loss: 0.0069
  
```

17. Visualise the Training set results on the **history** plot.

```

  ▾ Visualising the Training set results

  ▶ plt.plot(hist.history['loss'])
    plt.title('Training model loss')
    plt.ylabel('loss')
    plt.xlabel('epoch')
    plt.legend(['train'], loc='upper left')
    plt.show()
  
```



18. Import the **Test** dataset that you have downloaded by clicking on the File icon and selecting the file.
19. Perform preprocessing on the data by Cleaning the '**Close**' column as we did for the Training set.

▼ Importing the Test dataset

```
testData = pd.read_csv('Google_test_data.csv')
testData["Close"] = pd.to_numeric(testData.Close, errors='coerce')
testData = testData.dropna()
testData = testData.iloc[:, 4:5]
y_test = testData.iloc[60:, 0].values
```

20. Reshape the input to be in **Timesteps** which is required for Long Short-Term Memory (LSTM). Then, convert the data from a pandas dataframe to a numpy array.

▼ Adding the Timesteps

```
#input array for the model
inputClosing = testData.iloc[:, 0].values
inputClosing_scaled = sc.transform(inputClosing)
inputClosing_scaled.shape
X_test = []
length = len(testData)
timestep = 60
for i in range(timestep, length):
    X_test.append(inputClosing_scaled[i-timestep:i, 0])
X_test = np.array(X_test)
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))
X_test.shape
```

(192, 60, 1)

OUTPUT

1. Predict the Testing set results.

```

Predicting the test set

y_pred = rnn.predict(x_test)
y_pred

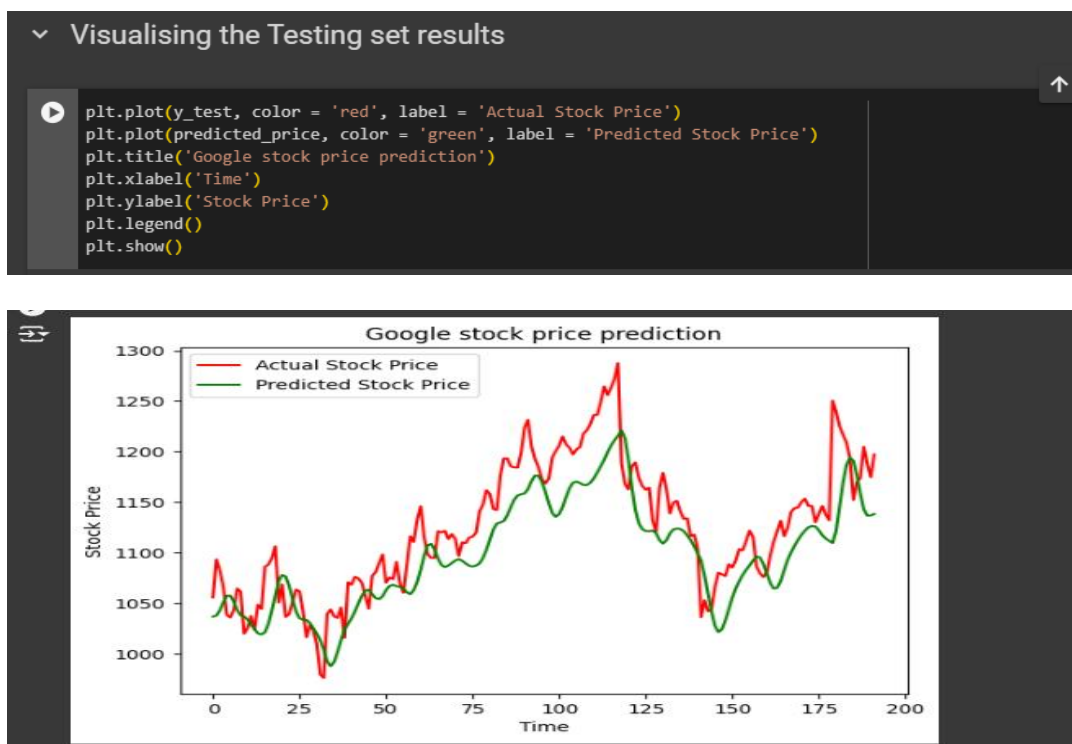
[1.3633572],
[1.3647286],
[1.3627164],
[1.3569885],
[1.34923 ],
[1.338233 ],
[1.3258127],
[1.3117162],
[1.2856901],
[1.252867 ],
[1.2186208],
[1.1889532],
[1.1712229],
[1.1690418],
[1.1794369],
[1.1965059],
[1.2167286],
[1.2358532],
[1.2523205],
[1.2669522],

[17]
[1.3389306],
[1.3552434],
[1.3910247],
[1.432737 ],
[1.4674519],
[1.4879729],
[1.4916408],
[1.4747498],
[1.4474221],
[1.4196421],
[1.4031538],
[1.3967783],
[1.3948383]], dtype=float32)

predicted_price = sc.inverse_transform(y_pred)

```

2. Visualize the Test set results on the **history** plot.



3. Run all the code cells to make sure there are no errors.

EXPERIMENT NO.: 4

NAME

Natural Language Processing (NLP) - Part of Speech (POS), Lemmatization and Stemming

OBJECTIVE

To perform Part of Speech (POS), Lemmatization and Stemming using SpaCy.

THEORY

Natural language processing (NLP) is a field of computer science and a subfield of artificial intelligence that aims to make computers understand human language. NLP uses **computational linguistics**, which is the study of how language works, and various models based on statistics, machine learning, and deep learning. These technologies allow computers to analyze and process text or voice data, and to grasp their full meaning, including the speaker's or writer's intentions and emotions.

NLP powers many applications that use language, such as text translation, voice recognition, text summarization, and chatbots. You may have used some of these applications yourself, such as voice-operated GPS systems, digital assistants, speech-to-text software, and customer service bots. NLP also helps businesses improve their efficiency, productivity, and performance by simplifying complex tasks that **involve language**.

Working in natural language processing (NLP) typically involves using **computational techniques** to analyze and understand human language. This can include tasks such as language understanding, language generation, and language interaction.

SpaCy

SpaCy is an open-source software library for **advanced natural language processing**, written in the programming languages Python and Cython. The library is published under the MIT license and its main developers are Matthew Honnibal and Ines Montani, the founders of the software company Explosion. SpaCy also supports deep learning workflows that allow connecting statistical models trained by popular machine learning libraries like TensorFlow and PyTorch.

Enabling machine to understand and process **raw text** is not easy. Same word plays differently in different context of a sentence. Sometime words which are completely different, tells almost the same meaning. Even splitting text into useful **word-like units** can be difficult in many languages. While it's possible to solve some problems starting from only the raw characters, it's usually better to use **linguistic knowledge** to add useful information. That's exactly what **spaCy** is designed to do: you put in raw text, and get back a Doc object, that comes with a variety of annotations.

Part of Speech (POS)

It is important to find named **entities** for text preprocessing in Natural Language Processing (NLP). It is a method of identifying words as **nouns, verbs, adjectives, adverbs**, etc.

Through the addition of a layer of **syntactic** and **semantic** information to the words, this procedure makes it easier to comprehend the sentence's structure and meaning.

Through improved comprehension of **phrase structure** and semantics, this technique makes it possible for machines to study and comprehend human language more accurately.

Lemmatization

It is a process of grouping together the inflected forms of a word so they can be analyzed as a **single** item, identified by the word's **lemma**, or dictionary form. It extracts the meaning of the word based on the dictionary. It basically extracts the **base word**.

Lemmatization is similar to stemming but it brings context to the words. So, it links words with similar meanings to one word. Lemmatization does **morphological analysis** of the words.

NLTK

Natural Language Toolkit (NLTK) is one of the largest Python libraries for performing various Natural Language Processing tasks. From rudimentary tasks such as text pre-processing to tasks like vectorized representation of text – **NLTK's API** has covered everything.

Stemming

Stemming refers to reducing a word to its **root form**. While performing natural language processing tasks, you will encounter various scenarios where you find different words with the same root. For instance, compute, computer, computing, computed, etc. You may want to reduce the words to their root form for the sake of uniformity. This is where stemming comes into play.

Named Entity Recognition (NER)

Named Entity Recognition is the most important starting step in **Information Retrieval**. Information Retrieval is the technique to extract important and useful information from unstructured raw text documents. Named Entity Recognition (NER) works by locating and identifying the named entities present in **unstructured text** into the standard categories such as person names, locations, organizations, time expressions, quantities, monetary values, percentage, codes etc. **SpaCy** comes with an extremely fast **statistical entity recognition system** that assigns labels to contiguous spans of tokens.

Stopwords

In natural language processing (NLP), stopwords are frequently filtered out to enhance **text analysis** and computational efficiency. Eliminating stopwords can improve the **accuracy** and relevance of NLP tasks by drawing attention to the more important words, or content words.

Certain words, like **“the,” “and,”** and **“is,”** are thought to be ineffective for communicating important information. The objective of eliminating words that add little or nothing to the comprehension of the text is to **expedite text processing**, even though the list of stopwords may differ.

A stop word is a commonly used word (such as **“the”, “a”, “an”,** or **“in”**) that a search engine has been programmed to **ignore**, both when indexing entries for searching and when retrieving them as the result of a search query.

We would not want these words to take up space in our database or take up valuable processing time. For this, we can **remove** them easily, by storing a list of words that you consider to stop words. NLTK (Natural Language Toolkit) in Python has a list of stopwords stored in **16** different languages.

Word Cloud

Word Cloud is a **data visualization** technique used for representing text data in which the size of each word indicates its frequency or importance. Significant textual data points can be highlighted using a word cloud. Word clouds are widely used for **analyzing data** from social network websites.

It is a visualization technique for text data wherein each word is **picturized** with its importance in the context or its frequency. For example, this is a very handy application when it comes to understanding the **crux** of today's news or the content of any youtube channel.

Advantages of Word Clouds :

1. Analyzing customer and employee feedback.
2. Identifying new Search Engine Optimization (SEO) keywords to target.

Drawbacks of Word Clouds :

1. Word Clouds are not perfect for every situation.
2. Data should be optimized for context.

PROCEDURE

1. Click on New notebook in Drive and Create a new project file.
2. Download the datasets : **Google_train_data.csv** and **Google_test_data.csv** on your device using the given drive link.
3. Edit the title of the File by clicking on the **‘Untitled0.ipynb’** text.
4. Create a new Text cell by clicking on the **‘+ Text’** symbol.
5. Create a new code cell by clicking on the **‘+ Code’** symbol.
6. Pip install the library - **SpaCy**.

```

v Natural Language Processing

v Importing the libraries

• spaCy: https://spacy.io/

[1] !pip install spacy

Requirement already satisfied: spacy in /usr/local/lib/python3.10/dist-packages (3.7.6)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.10/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.10/dist-packages (from spacy) (1.0.10)
Requirement already satisfied: cyemem<2.1.0,>=2.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy) (2.0.8)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy) (3.0.9)
Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /usr/local/lib/python3.10/dist-packages (from spacy) (8.2.5)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.10/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.10/dist-packages (from spacy) (2.4.8)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.10/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.1.0 in /usr/local/lib/python3.10/dist-packages (from spacy) (0.4.1)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.10/dist-packages (from spacy) (0.12.5)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.10/dist-packages (from spacy) (4.66.5)

```

7. Import the necessary libraries like :-
 - a) **SpaCy** - which is an open-source software library which is used for Advanced Natural Language Processing (NLP).
 - b) **Bs4 (Beautiful Soup)** - used for pulling data out of HTML and XML files which makes it easy to scrape information from web pages.
 - c) **Urllib.request** - which is used to fetch URLs (Uniform Resource Locators).
 - d) **Nltk (Natural Language Toolkit)** - which is an open-source software library which is used for Natural Language Processing (NLP).
8. Make sure you are using the latest version of SpaCy. To check the version, type 'spacy.__version__' as below.

```

[2] import bs4 as bs # Beautiful soup
import urllib.request
import nltk
import spacy
spacy.__version__

'3.7.6'

```

9. Download the Portuguese, French and English language packages for the implementations using the SpaCy library.

```

!python -m spacy download pt_core_news_sm #Portuguese

Collecting pt-core-news-sm==3.7.0
  Downloading https://github.com/explosion/spacy-models/releases/download/pt\_core\_news\_sm-3.7.0/pt\_core\_news\_sm-3.7.0.tar.gz
    13.0/13.0 MB 47.7 MB/s eta 0:00:00
Requirement already satisfied: spacy<3.8.0,>=3.7.0 in /usr/local/lib/python3.10/dist-packages (from pt-core-news-sm==3.7.0) (3.7.6)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (1.0.10)
Requirement already satisfied: cyemem<2.1.0,>=2.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (2.0.8)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (3.0.9)
Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (8.2.5)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (2.4.8)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.1.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (0.4.1)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (0.12.5)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (4.66.5)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0) (2.28.1)

```

```

!python -m spacy download fr_core_news_sm #French
Collecting fr-core-news-sm==3.7.0
  Downloading https://github.com/explosion/spacy-models/releases/download/fr_core_news_sm-3.7.0-16.3/16.3 MB 57.7 MB/s eta 0:00:00
Requirement already satisfied: spacy<3.8.0,>=3.7.0 in /usr/local/lib/python3.10/dist-packages (from fr-core-news-sm==3.7.0)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: pydantic<1.8.1,>=1.8.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)
Requirement already satisfied: packaging<20.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.0)

```

```

!python -m spacy download en_core_web_sm #English
Collecting en-core-web-sm==3.7.1
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.7.1/12.8/12.8 MB 52.9 MB/s eta 0:00:00
Requirement already satisfied: spacy<3.8.0,>=3.7.2 in /usr/local/lib/python3.10/dist-packages (from en-core-web-sm==3.7.1)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: thinc<8.3.0,>=8.2.2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: weasel<0.5.0,>=0.1.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: pydantic<1.8.1,>=1.8.1 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)
Requirement already satisfied: packaging<20.0 in /usr/local/lib/python3.10/dist-packages (from spacy<3.8.0,>=3.7.2)

```

10. Import the English language toolkit and load it using spacy.
11. Perform '**Part-of-Speech**' on the token in the document to identify the words in the sentence as a noun, verb, adjective or adverb.

```

v POS (part-of-speech)
  • POS (part-of-speech): noun, adjective, verb
  • It is important to find named entities
  • Tags: https://ashutoshtripahty.com/2020/04/13/parts-of-speech-tagging-and-dependency-parsing-using-spacy/

[6] import en_core_web_sm

[7] nlp = spacy.load('en_core_web_sm')

[8] nlp
<spacy.lang.en.English at 0x7cd5526431c0>

[9] document = nlp('I am learning natural language processing. The course is in London')

```

```

for token in document:
    print(token.text, token.pos_)

I PRON
am AUX
learning VERB
natural ADJ
language NOUN
processing NOUN
. PUNCT
The DET
course NOUN
is AUX
in ADP
London PROPN

```

12. Print the token text of the document and compare it with the following text preprocessing techniques :-
 - i. **lemma**: "root" of the word.
 - ii. **pos**: part-of-speech.

- iii. **tag**: morfological information (present, future, past).
- iv. **dep**: syntatic dependency.
- v. **shape**: lowercase, uppercase.
- vi. **alpha**: if it is alphanumeric.
- vii. **stop**: if it is a stop word.

Legend

- lemma: "root" of the word
- pos: part-of-speech
- tag: morfological information (present, future, past)
- dep: syntatic dependency
- shape: lowercase, uppercase
- alpha: if it is alphanumeric
- stop: if it is a stop word

```
[11] for token in document:
      print(token.text, token.pos_, token.lemma_, token.tag_, token.dep_, token.shape_, token.is_alpha, token.is_stop)
```

```
I PRON I PRP nsubj X True True
am AUX be VBP aux xx True True
learning VERB learn VBG ROOT xxxx True False
natural ADJ natural JJ amod xxxx True False
language NOUN language NN compound xxxx True False
processing NOUN processing NN dobj xxxx True False
. PUNCT . . punct . False False
The DET the DT det Xxx True True
course NOUN course NN nsubj xxxx True False
is AUX be VBZ ROOT xx True True
in ADP in IN prep xx True True
London PROPN London NNP pobj Xxxxx True False
```

```
for token in document:
    if token.pos_ == 'PRON':
        print(token.text)
```

```
I
```

```
[13] for token in document:
      if token.pos_ == 'VERB':
          print(token.text)
```

```
learning
```

13. Perform '**Lemmatization**' on the token in the document to extract the base word and identify the meaning of the word based on the dictionary with the help of **morphological analysis** using SpaCy.

Lemmatization and stemming

- Lemmatization: meaning of the word based on the dictionary (morphological analysis) - extract the base word
- Stemming: extract the root of the word

```
for token in document:
    print(token.text, token.lemma_)
```

```
I I
am be
learning learn
natural natural
language language
processing processing
...
The the
course course
is be
in in
London London
```

```
[15] doc = nlp('learn learning watch watching watched')
      [token.lemma_ for token in doc]
```

```
['learn', 'learn', 'watch', 'watch', 'watch']
```

14. Perform '**Stemming**' on the token in the document to extract the root of the word for the sake of uniformity with the help of the '**PorterStemmer**' function using Natural Language Toolkit (NLTK).

```

  ✓ [16] import nltk

  ✓ [17] stemmer = nltk.stem.PorterStemmer()
        stemmer.stem('learning')
        'learn'

  ✓ [18] stemmer.stem('watching')
        'watch'

  for token in document:
    print(token.text, token.lemma_, stemmer.stem(token.text))

  I I i
  am be am
  learning learn learn
  natural natural natur
  language language languag
  processing processing process
  . . .
  The the the
  course course cours
  is be is
  in in in
  London London london

```

15. Load texts from Wikipedia on the internet using the '**urllib.request**' library and providing the path.

```

  ✓ [20] data = urllib.request.urlopen('https://en.wikipedia.org/wiki/Artificial_intelligence')

  ✓ [21] data = data.read()
        data

  <!DOCTYPE html>
  <html class="client-nojs vector-feature-language-in-header-enabled vector-feature-language-in-main-page-header-disabled vector-feature-sticky-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-enabled vector-feature-appearance-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector-toc-available" dir="ltr" lang="en">
  <head>
  <meta charset="utf-8"/>
  <title>Artificial intelligence - Wikipedia</title>
  <script>(function(){var className="client-js vector-feature-language-in-header-enabled vector-feature-language-in-main-page-header-disabled vector-feature-sticky-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-enabled vector-feature-appearance-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector-toc-available";document.documentElement.className=className;})();RLCONF=
  {
    "wgBreakFrames":false,"wgSeparatorTransformTable":["",""],"wgDigitTransformTable":
    ["\n",""],"wgDefaultDateFormat":"dmy","wgMonthNames":
    [
      "",
      "January",
      "February",
      "March",
      "April",
      "May",
      "June",
      "July",
      "August",
      "September",
      "October",
      "November",
      "December"
    ],
    "wgRe
    f6f5-4aea-9640-
    818ea6e874fc",
    "wgCanonicalNamespace":"","wgCanonicalSpecialPageName":false,"wgNamespaceNumber":0,"wgPageName":"Artificia
    intelligence",
    "wgCurRevisionId":1244187555,"wgRevisionId":1244187555,"wgArticleId":1164,"wgIsArticle":true,"wgIsRedirect
    [false],"wgCategories":["Webarchive template wayback links","CS1: long volume value","Articles with short
    description","Short description is different from Wikidata","Use day dates from July 2023","Wikipedia indefinitely semi-
    protected pages","All articles with unsourced references","Articles with unsourced references from June 2023"]
  }

```

16. Process the text which was loaded to extract information out of the **HTML** tags using the '**Bs4** (Beautiful Soup)' library.

```

  ✓ [22] data_html = bs.BeautifulSoup(data, 'lxml')
        data_html

  <!DOCTYPE html>
  <html class="client-nojs vector-feature-language-in-header-enabled vector-feature-language-in-main-page-header-disab
  le vector-feature-sticky-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-pinned-clientpref-1
  vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-limited-width-conte
  nable vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-enabled vector-feature-appearance-pinne
  d clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector-toc-available" dir="ltr" lang="en">
  <head>
  <meta charset="utf-8"/>
  <title>Artificial intelligence - Wikipedia</title>
  <script>(function(){var className="client-js vector-feature-language-in-header-enabled vector-feature-language-in-main
  page-header-disabled vector-feature-sticky-header-disabled vector-feature-page-tools-pinned-disabled vector-feature-to
  pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-limited-width-clientpref-1 vector-feature-
  limited-width-content-enabled vector-feature-custom-font-size-clientpref-1 vector-feature-appearance-enabled vector-
  feature-appearance-pinned-clientpref-1 vector-feature-night-mode-enabled skin-theme-clientpref-day vector-toc-
  available";document.documentElement.className=className;})();RLCONF=
  {
    "wgBreakFrames":false,"wgSeparatorTransformTable":["",""],"wgDigitTransformTable":
    ["\n",""],"wgDefaultDateFormat":"dmy","wgMonthNames":
    [
      "",
      "January",
      "February",
      "March",
      "April",
      "May",
      "June",
      "July",
      "August",
      "September",
      "October",
      "November",
      "December"
    ],
    "wgRe
    f6f5-4aea-9640-
    818ea6e874fc",
    "wgCanonicalNamespace":"","wgCanonicalSpecialPageName":false,"wgNamespaceNumber":0,"wgPageName":"Artificia
    intelligence",
    "wgCurRevisionId":1244187555,"wgRevisionId":1244187555,"wgArticleId":1164,"wgIsArticle":true,"wgIsRedirect
    [false],"wgCategories":["Webarchive template wayback links","CS1: long volume value","Articles with short
    description","Short description is different from Wikidata","Use day dates from July 2023","Wikipedia indefinitely semi-
    protected pages","All articles with unsourced references","Articles with unsourced references from June 2023"]
  }

```

17. Extract only the text from the first paragraph and store it in the variable 'content'.

```
[23] paragraphs = data_html.find_all('p')

len(paragraphs)

167

[25] paragraphs[1]

<p><b>Artificial intelligence</b> (AI), in its broadest sense, is intelligence exhibited by machines, particularly computer systems. It is a field of research in computer science that develops and studies methods and software that enable machines to perceive their environment and use learning and intelligence to take actions that maximize their chances of achieving defined goals. Some high-profile applications of AI include advanced web search engines (e.g., Google Search); recommendation systems (used by YouTube, Amazon, and Netflix); interacting via human speech (e.g., Google Assistant, Siri, and Alexa); autonomous vehicles (e.g., Waymo); generative and creative tools (e.g., ChatGPT, Apple Intelligence, and AI art); and superhuman play and analysis in strategy games (e.g., chess and Go). However, many AI applications are not perceived as AI: "A lot of cutting edge AI has filtered into general applications, often without being called AI because once something is common, it is no longer special."

[26] paragraphs[1].text

'Artificial intelligence (AI), in its broadest sense, is intelligence exhibited by machines, particularly computer systems. It is a field of research in computer science that develops and studies methods and software that enable machines to perceive their environment and use learning and intelligence to take actions that maximize their chances of achieving defined goals.[1] Such machines may be called AIs. Some high-profile applications of AI include advanced web search engines (e.g., Google Search); recommendation systems (used by YouTube, Amazon, and Netflix); interacting via human speech (e.g., Google Assistant, Siri, and Alexa); autonomous vehicles (e.g., Waymo); generative and creative tools (e.g., ChatGPT, Apple Intelligence, and AI art); and superhuman play and analysis in strategy games (e.g., chess and Go). However, many AI applications are not perceived as AI: "A lot of cutting edge AI has filtered into general applications, often without being called AI because once something is common, it is no longer special."'

content = ''
for p in paragraphs:
    content += p.text

[28] content

'\nArtificial intelligence (AI), in its broadest sense, is intelligence exhibited by machines, particularly computer systems. It is a field of research in computer science that develops and studies methods and software that enable machines to perceive their environment and use learning and intelligence to take actions that maximize their chances of achieving defined goals.[1] Such machines may be called AIs. Some high-profile applications of AI include advanced web search engines (e.g., Google Search); recommendation systems (used by YouTube, Amazon, and Netflix); interacting via human speech (e.g., Google Assistant, Siri, and Alexa); autonomous vehicles (e.g., Waymo); generative and creative tools (e.g., ChatGPT, Apple Intelligence, and AI art); and superhuman play and analysis in strategy games (e.g., chess and Go). However, many AI applications are not perceived as AI: "A lot of cutting edge AI has filtered into general applications, often without being called AI because once something is common, it is no longer special."'

content

'\nArtificial intelligence (AI), in its broadest sense, is intelligence exhibited by machines, particularly computer systems. It is a field of research in computer science that develops and studies methods and software that enable machines to perceive their environment and use learning and intelligence to take actions that maximize their chances of achieving defined goals.[1] Such machines may be called AIs. Some high-profile applications of AI include advanced web search engines (e.g., Google Search); recommendation systems (used by YouTube, Amazon, and Netflix); interacting via human speech (e.g., Google Assistant, Siri, and Alexa); autonomous vehicles (e.g., Waymo); generative and creative tools (e.g., ChatGPT, Apple Intelligence, and AI art); and superhuman play and analysis in strategy games (e.g., chess and Go). However, many AI applications are not perceived as AI: "A lot of cutting edge AI has filtered into general applications, often without being called AI because once something is common, it is no longer special."'

[29] content = content.lower()
content

'\nartificial intelligence (ai), in its broadest sense, is intelligence exhibited by machines, particularly computer systems. It is a field of research in computer science that develops and studies methods and software that enable machines to perceive their environment and use learning and intelligence to take actions that maximize their chances of achieving defined goals.[1] Such machines may be called aIs. Some high-profile applications of ai include advanced web search engines (e.g., google search); recommendation systems (used by youtube, amazon, and netflix); interacting via human speech (e.g., google assistant, siri, and alexa); autonomous vehicles (e.g., waymo); generative and creative tools (e.g., chatgpt, apple intelligence, and ai art); and superhuman play and analysis in strategy games (e.g., chess and go). However, many ai applications are not perceived as ai: "A lot of cutting edge ai has filtered into general applications, often without being called ai because once something is common, it is no longer special."'

```

18. Perform Text search of the string name 'turing' in the document using the 'PhraseMatcher' function and you will get the strings of the matches.

```
Text search with spaCy

[30] string = 'turing'
search_token = nlp(string)

[31] nlp.vocab

<spacy.vocab.Vocab at 0x7cd484dcaa70>

from spacy.matcher import PhraseMatcher
matcher = PhraseMatcher(nlp.vocab)
matcher.add('SEARCH', None, search_token)

```

```

[33] doc = nlp(content)
     matches = matcher(doc)
     matches

[(8661325627334373315, 2622, 2623),
 (8661325627334373315, 10001, 10002),
 (8661325627334373315, 10718, 10719),
 (8661325627334373315, 10808, 10809),
 (8661325627334373315, 10823, 10824),
 (8661325627334373315, 11837, 11838),
 (8661325627334373315, 11884, 11885),
 (8661325627334373315, 11929, 11930),
 (8661325627334373315, 11963, 11964)]

[34] doc[2622:2623], doc[2622-5:2623+5]

(turing, programming language prolog, is turing complete. moreover, its)

[35] doc[10001:10002], doc[10001-5:10002+5]

(turing, —developed by the alan turing institute tests projects in four)

[36] matches[0], matches[0][0], matches[0][1], matches[0][2]

((8661325627334373315, 2622, 2623), 8661325627334373315, 2622, 2623)

```

```

from IPython.core.display import HTML
text = ''
number_of_words = 50
doc = nlp(content)
matches = matcher(doc)

display(HTML(f'<h1>{string.upper()}</h1>'))
display(HTML(f'<p><strong>Search results:</strong> {len(matches)}</p>'))
for i in matches:
    start = i[1] - number_of_words
    if start < 0:
        start = 0
    text += str(doc[start:i[2] + number_of_words]).replace(string, f'<mark>{string}</mark>')
    text += "<br /><br />"

display(HTML(f'<pre>...{text} ... </pre>'))

```

TURING

Search results: 9

...premises that include the negation of the problem to be solved [83] inference in both horn clause logic and first-order logic is undecidable, and therefore intractable. however, backward reasoning with horn clauses, which underpins computation in the logic programming language prolog, is complete. moreover, its efficiency is competitive with computation in other symbolic programming languages [84] fuzzy logic assigns a "degree of truth" between 0 and 1. it can therefore handle propositions that are vague and partially true [85] non-monotonic logics, including

pre-release audits and cost-benefit analyses [274] artificial intelligence projects can have their ethical permissibility tested while designing, developing, and implementing an ai system. an ai framework such as the care and act framework containing the sum values—developed by the alan turing institute tests projects in four main areas [275][276] other developments in ethical frameworks include those decided upon during the asilomar conference, the montreal declaration for responsible ai, and the lee's ethics of autonomous systems initiative, among others [277] however, these principles do not go

in may 2024 at the ai seoul summit, 16 global ai tech companies agreed to safety commitments on the development of ai [295][296] the study of mechanical or "formal" reasoning began with philosophers and mathematicians in antiquity. the study of logic led directly to alan turing's theory of computation, which suggested that a machine, by shuffling symbols as simple as "0" and "1", could simulate any conceivable form of mathematical reasoning [297][298] this, along with concurrent discoveries in cybernetics, information theory and neurobiology, led researchers

discoveries in cybernetics, information theory and neurobiology, led researchers to consider the possibility of building an "electronic brain" [r] they developed several areas of research that would become part of ai [300] such as mcculloch and pitts design for "artificial neurons" in 1943 [115] and turing's influential 1950 paper 'computing machinery and intelligence', which introduced the turing test and showed that "machine intelligence" was plausible [301][298] the field of ai research was founded at a workshop at dartmouth college in 1956 [s][6] the attendees became the leaders of ai

of building an "electronic brain" [r] they developed several areas of research that would become part of ai [300] such as mcculloch and pitts design for "artificial neurons" in 1943 [115] and turing's influential 1950 paper 'computing machinery and intelligence', which introduced the turing test and showed that "machine intelligence" was plausible [301][298] the field of ai research was founded at a workshop at dartmouth college in 1956 [s][6] the attendees became the leaders of ai research in the 1960s [t] they and their students produced programs that the press described

19. To perform the Named-Entity Recognition (NER) print the 'Entity label'. To highlight the Entities, import 'displacy' from the spacy library.

```

v Named entity recognition

• NER (Named-Entity Recognition)
• List of tags: https://towardsdatascience.com/named-entity-recognition-ner-using-spacy-nlp-part-4-28da2ece57c6

[37] for entity in doc.ents:
     print(entity.text, entity.label_)

2012 DATE
2017 DATE
the 2020s hundreds of billions of dollars DATE
the 21st century DATE
the late 1980s and 1990s DATE
first ORDINAL
two CARDINAL
one CARDINAL
english LANGUAGE
margaret PERSON
thesauri ORG
2019 DATE
2023 DATE
mood-165 ORG
two CARDINAL
two CARDINAL
two CARDINAL
two CARDINAL
first ORDINAL

```

```
[39] for entity in doc.ents:
      if entity.label_ == 'ORG':
          print(entity.text, entity.label_)

ais ORG
google ORG
google ORG
thesauri ORG
mood.[65] ORG
object.[111] ORG
gpt ORG
gpt ORG
subword ORG
gpt ORG
gpt ORG
gpt ORG
google ORG
adas ORG
microsoft ORG
google ORG
apple ORG
microsoft ORG
google ORG
cao ORG
ibm ORG
google deepmind ORG
```

```
from spacy import displacy
displacy.render(doc, style='ent', jupyter=True)

thus discrediting the approach altogether[311] the "ai winter", a period when obtaining funding for ai projects was difficult, followed[9]
in the early 1980s DATE , ai research was revived by the commercial success of expert systems.[312] a form of ai program that simulated the
knowledge and analytical skills of human experts. by 1985 DATE , the market for ai had reached over a billion dollars MONEY . at the same
time, japan GPE 's fifth ORDINAL generation computer project inspired the u.s. GPE and british NORP governments to restore
funding for academic research.[8] however, beginning with the collapse of the lisp machine market in 1987 DATE , ai once again fell into
disrepute, and a second ORDINAL longer-lasting winter began.[10]
up to this point, most of ai's funding had gone to projects that used high-level symbols to represent mental objects like plans, goals, beliefs, and
known facts. in the 1980s DATE , some researchers began to doubt that this approach would be able to imitate all the processes of human
cognition, especially perception, robotics, learning and pattern recognition.[313] and began to look into "sub-symbolic" approaches.[314] rodney
brooks rejected "representation" in general and focussed directly on engineering machines that move and survive.[x] judea pearl PERSON , lofi
```

20. To visualize the most terms that are in the article from **Wordcloud**, we need to get rid of all the stopwords first.

```
Wordcloud and stop words

[41] from spacy.lang.en.stop_words import STOP_WORDS
      print(STOP_WORDS)

{'around', 'per', 'name', 'you', 'without', 'somehow', 'must', 'full', 'might', 'whole', 'it', 'sometime', 'not', 'something'}

[42] len(STOP_WORDS)
326

[43] doc = nlp(content)
      token_list = []
      for token in doc:
          token_list.append(token.text)

print(token_list)

['\n', 'artificial', 'intelligence', '(', 'ai', ')', ',', 'in', 'its', 'broadest', 'sense', ',', 'is', 'intelligence', 'ex']
```

21. To get rid of all the **Stopwords**, create an empty list. Then, append all the words that are not stopwords to this list.

```
[45] len(token_list)
14224

[46] nlp.vocab['will'].is_stop
True

no_stop = []
for word in token_list:
    if nlp.vocab[word].is_stop == False:
        no_stop.append(word)

[48] len(no_stop)
9010
```

```
[45] len(token_list)
14224
```

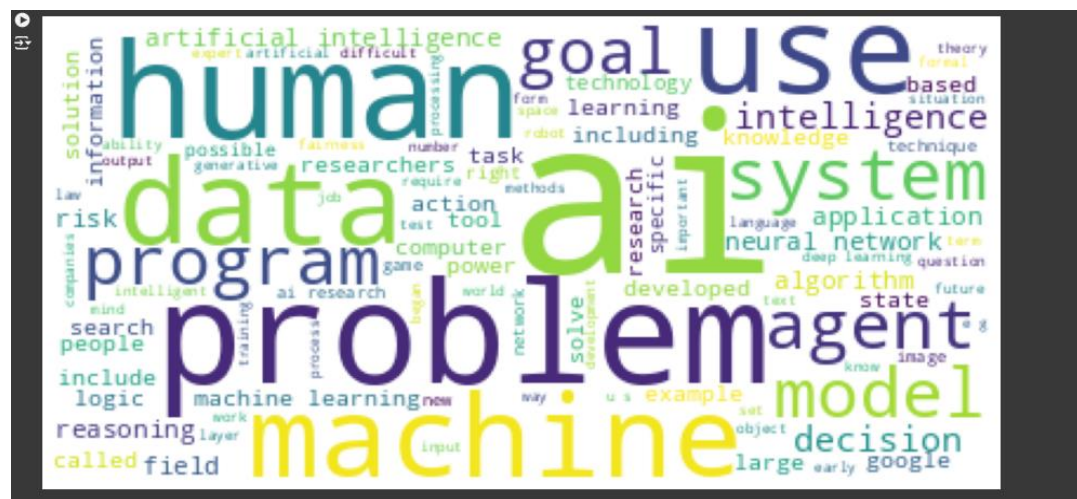
```
[46] nlp.vocab['will'].is_stop
True
```

```
no_stop = []
for word in token_list:
    if nlp.vocab[word].is_stop == False:
        no_stop.append(word)
```

```
[48] len(no_stop)
9810
```

22. Create the Wordcloud using this list.

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
cloud = WordCloud(background_color = 'white', max_words = 100)
cloud = cloud.generate(' '.join(no_stop))
plt.figure(figsize = (15,15))
plt.imshow(cloud)
plt.axis('off')
plt.show()
```



23. Run all the cells to make sure there are no errors.

EXPERIMENT NO.: 5

NAME

Natural Language Processing (NLP) – Sentiment Analysis

OBJECTIVE

To perform Sentiment Analysis using SpaCy on a Twitter dataset.

THEORY

Sentiment analysis is a popular task in natural language processing. The goal of sentiment analysis is to classify the text based on the mood or mentality expressed in the text, which can be positive negative, or neutral.

Sentiment analysis is the process of analyzing digital text to determine if the emotional tone of the message is **positive**, **negative**, or **neutral**. Today, companies have large volumes of text data like emails, customer support chat transcripts, social media comments, and reviews. Sentiment analysis tools can scan this text to automatically determine the author's attitude towards a topic. Companies use the insights from sentiment analysis to improve customer service and increase brand reputation.

Sentiment analysis, also known as opinion mining, is an important **business intelligence tool** that helps companies improve their products and services.

Sentiment analysis is the contextual meaning of words that indicates the **social sentiment** of a brand and also helps the business to determine whether the product they are manufacturing is going to make a demand in the market or not.

According to the survey, **80%** of the world's data is **unstructured**. The data needs to be analyzed and be in a structured manner whether it is in the form of emails, texts, documents, articles, and many more.

1. Sentiment Analysis is required as it stores data **efficiently** and is **cost friendly**.
2. Sentiment analysis solves **real-time** issues and can help you solve all real-time scenarios.

Data preprocessing

Starting with collecting the text data that needs to be analysed for sentiment like customer reviews, social media posts, news articles, or any other form of textual content. The collected text is pre-processed to clean and standardize the data with various tasks:

1. Removing irrelevant information (e.g., HTML tags, special characters).
2. Tokenization: Breaking the text into individual words or tokens.
3. Removing stop words (common words like "and," "the," punctuation marks etc. that don't contribute much to sentiment).
4. Stemming or Lemmatization: Reducing words to their root form.

Sentiment Analysis with NLTK

Sentiment Analysis is the process of analyzing **textual data** to determine the sentiment expressed within. This can range from simple binary classification (positive or negative) to more complex systems that can detect specific emotions or levels of sentiment. The primary goal is to understand the subjective information in the text.

NLTK (Natural Language Toolkit) is a powerful library in Python that provides easy-to-use interfaces to over **50 corpora** and **lexical** resources, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning.

VADER (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon and rule-based sentiment analysis tool that is specifically attuned to sentiments expressed in social media. A sentiment lexicon is a Polarity Classifier. It is a list of lexical features (e.g., words) which are generally labeled according to their semantic orientation as either positive or negative. VADER not only tells about the Positivity and Negativity score but also tells us the magnitude (score) of how positive or negative a sentiment is.

Sentiment Analysis with SpaCy

SpaCy provides more robust, efficient pipelines and features when compared to others like TectBlob, NLTK etc. Now they have added the capability to incorporate transformers also into their module for training and inference purposes. Unfortunately, SpaCy doesn't have an inbuilt sentiment analysis module, although fortunately, they have a text classification module that can be custom trained and be served as a model.

All you need to do is pick a dataset, label and configure them as per SpaCy's format, pick a base model configuration and that's all. Training a spaCy pipeline is as easy now as ever, we need the data in the required format, select the base model configuration and train your pipeline.

PROCEDURE

1. Click on New notebook in Drive and Create a new project file.
2. Download the datasets : **Google_train_data.csv** and **Google_test_data.csv** on your device using the given drive link.
3. Edit the title of the File by clicking on the '**Untitled0.ipynb**' text.
4. Create a new Text cell by clicking on the '**+ Text**' symbol.
5. Create a new code cell by clicking on the '**+ Code**' symbol.
6. Pip install and upgrade the library - **SpaCy**.
7. Download the English language package for the implementations using the SpaCy library.
8. Import the necessary libraries like :-
 - a. **SpaCy** - which is an open-source software library which is used for Advanced Natural Language Processing (NLP).
 - b. **Nltk (Natural Language Toolkit)** - which is an open-source software library which is used for Natural Language Processing (NLP).
 - c. **en_core_web_sm** - which is used to import the English language package.
 - d. **Pandas** - which is used for data manipulation and analysis.
 - e. **Seaborn** – which is used for making statistical graphs.
 - f. **Numpy** - which is used for working with arrays.

- g. **Re (Regular Expressions) -**
- h. **Random -**

OUTPUT